



Expense Tracker

Gestor de gastos personales con Angular 21 y Firebase
Informe técnico

AUTOR	Pablo Nicolás Armijos Sánchez
DEMO	expense-tracker-pablo.vercel.app
REPOSITORIO	github.com/pabloarmijos11/expense-tracker
STACK	Angular 21.2 · Signal Forms · Firebase 12 · Tailwind CSS 4 · Vitest · GitHub Actions · Vercel
LICENCIA	MIT
FECHA	21 de septiembre de 2026

1. Contexto y objetivos

Expense Tracker permite registrar gastos con un desglose opcional por líneas, clasificarlos en categorías propias y fijar presupuestos mensuales por categoría para saber cuánto queda del mes. Cada usuario ve únicamente sus datos.

El proyecto se construyó para practicar a fondo cuatro terrenos concretos de Angular 21, y terminó enseñando algo que no estaba en el plan: un fallo de pruebas que obligó a cambiar cómo la aplicación recibe sus dependencias. Esa sección (la 5) es la parte más útil de este informe.

Objetivo	Cómo se cumple
Formularios complejos	Signal Forms , la API que estrena Angular 21: líneas dinámicas, validación cruzada y validación asíncrona contra la base de datos.
Routing avanzado	Rutas anidadas, tres guards con propósitos distintos y un resolver que carga el gasto antes de entrar.
Carga diferida	La sección de presupuestos viaja en su propio paquete, con el guard colocado donde de verdad evita la descarga.
Datos cerrados por dueño	Reglas de Firestore con <code>ownerId</code> validado en cada operación e inmutable tras la creación.
Pruebas que verifiquen de verdad	66 pruebas que no tocan la red, tras rehacer la forma de sustituir el SDK de Firebase (sección 5).

2. Arquitectura y navegación

La aplicación se organiza por dominio: cada carpeta reúne el servicio, los componentes y las pruebas de una misma parcela del problema. El acceso a Firestore no se dispersa por los componentes; vive en un servicio por colección.

Carpeta	Contenido
<code>core/tokens</code>	Los tokens de inyección de Firebase y sus dobles para pruebas. El centro de la sección 5.
<code>core/guards</code>	<code>authGuard</code> , <code>guestGuard</code> y <code>unsavedChangesGuard</code> .
<code>core/resolvers</code>	<code>expenseResolver</code> : carga el gasto antes de construir la pantalla.
<code>expenses</code>	Lista, detalle y formulario, más <code>ExpenseService</code> .
<code>categories</code> · <code>budgets</code>	Una carpeta por dominio. Presupuestos se carga de forma diferida.

2.1 Tres guards que no hacen lo mismo

Guard	Qué impide
authGuard	Entrar a una zona privada sin sesión.
guestGuard	Volver al login o al registro <i>teniendo</i> sesión abierta.
unsavedChangesGuard	Abandonar el formulario con cambios sin guardar. Es un <code>canDeactivate</code> : actúa al salir, no al entrar.

2.2 Dónde se coloca el guard de una sección diferida

La sección de presupuestos se carga con `loadChildren` . El guard que la protege está en la ruta *padre*, y no es un detalle de estilo:

```
{ path: 'budgets',  
  canActivate: [authGuard], // aquí sí sirve  
  loadChildren: () => import('./budgets/budgets.routes') }
```

Un guard escrito dentro del archivo de rutas diferido no evitaría la descarga del paquete: para leer esas rutas el navegador ya ha tenido que bajarlo. Colocarlo en el padre es lo que hace que un visitante sin sesión no llegue siquiera a pedir el código de una sección que no puede ver.

3. Signal Forms: el núcleo técnico

Angular 21 introduce `@angular/forms/signals`, una forma de declarar formularios donde el estado es una señal y las reglas se describen sobre la forma del modelo. El formulario de gasto utiliza los tres casos que la hacen interesante.

3.1 Un array de campos que crece y mengua

Un gasto puede desglosarse en líneas. `applyEach` aplica las mismas reglas a cada elemento del array, exista uno o veinte, sin escribir la validación por índice.

```
applyEach(s.lines, (line) => {
  required(line.label, { message: 'La línea necesita una etiqueta' });
  min(line.amount, 0.01, { message: 'El monto debe ser mayor a 0' });
});
```

3.2 Validación cruzada: las líneas contra el total

Si el gasto tiene desglose, la suma de las líneas debe coincidir con el total. La regla no pertenece a ningún campo aislado, sino a la relación entre dos: `valueOf` permite leer otra parte del modelo desde la validación de un campo.

```
validate(s.total, ({ value, valueOf }) => {
  const lines = valueOf(s.lines);
  if (lines.length === 0) return undefined; // sin desglose, nada que cuadrar

  const sum = lines.reduce((acc, line) => acc + line.amount, 0);
  return Math.abs(sum - value()) > 0.01
    ? { kind: 'linesMismatch',
        message: `Las líneas suman ${sum.toFixed(2)}, pero el total es ${value()}` }
    : undefined;
});
```

La tolerancia de 0.01 no es arbitraria. Comparar sumas de decimales con `===` falla por cómo se representan los números en coma flotante: tres líneas de `0.1` no suman exactamente `0.3`. Sin ese margen, el formulario rechazaría gastos correctos y el usuario no tendría forma de entender por qué. Hay una prueba dedicada exclusivamente a ese caso.

3.3 Validación asíncrona contra la base de datos

Dos categorías no pueden llamarse igual. Comprobarlo exige preguntar a Firestore, así que la validación es asíncrona: consulta por dueño y nombre, pidiendo un solo documento —basta saber si existe alguno, no cuántos hay.

```
const duplicateQuery = this.sdk.query(
  this.categoriesCollection,
  this.sdk.where('ownerId', '==', user.uid),
  this.sdk.where('name', '==', trimmedName),
  this.sdk.limit(1), // existe o no; el número da igual
);
return !(await this.sdk.getDocs(duplicateQuery)).empty;
```

Nótese que las funciones del SDK llegan por `this.sdk` y no por un import. El porqué es toda la sección 5.

4. Modelo de datos y reglas de seguridad

Tres colecciones planas, cada documento con el identificador de su dueño. A diferencia de un modelo anidado por usuario, aquí la propiedad es un *campo*, no la ruta — lo que obliga a que las reglas lo comprueben en cada operación y, sobre todo, a impedir que ese campo cambie.

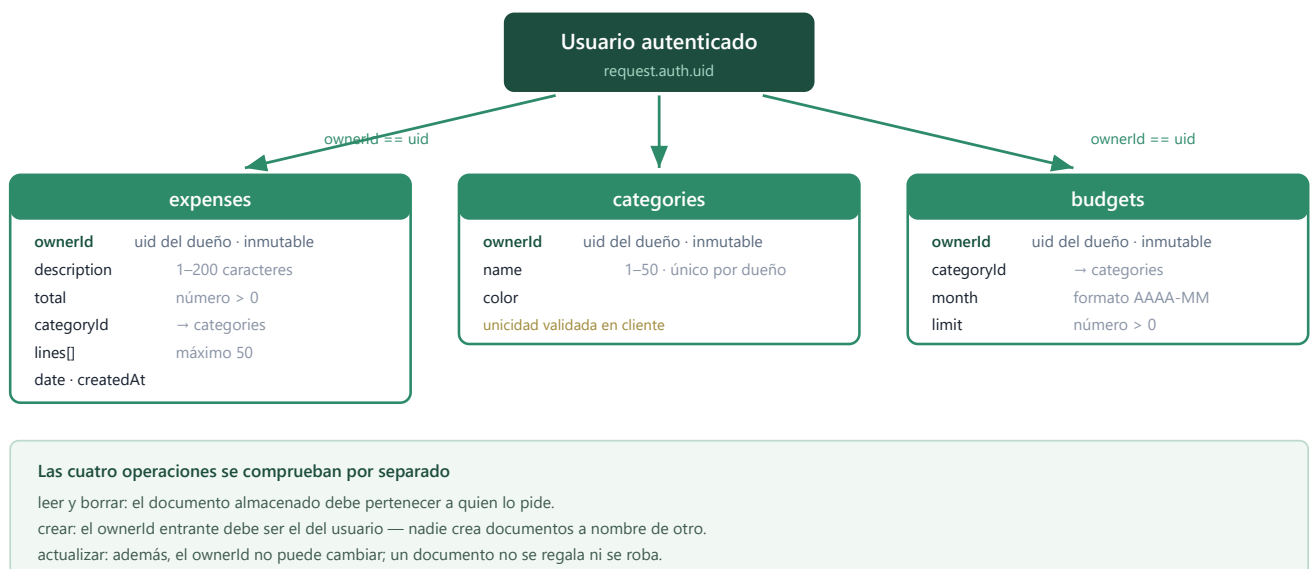


Figura 1 — Modelo de datos. La propiedad es un campo validado en cada operación, no una ruta.

```
function isOwner(data) {
  return isSignedIn() && data.ownerId == request.auth.uid;
}

match /expenses/{expenseId} {
  allow read: if isOwner(resource.data);
  allow create: if isSignedIn() &&
    request.resource.data.ownerId == request.auth.uid &&
    isValidExpense(request.resource.data);
  allow update: if isOwner(resource.data) &&
    isValidExpense(request.resource.data) &&
    ownerUnchanged(); // el dueño no cambia jamás
  allow delete: if isOwner(resource.data);
}
```

Cada validador usa `hasOnly` sobre la lista de campos permitidos: un documento con un campo de más se rechaza. Esto cierra la puerta a que un cliente modificado engorde los documentos con datos que la aplicación nunca lee pero que ocupan cuota.

5. El fallo que cambió la arquitectura de la aplicación

El primer *pull request* con integración continua reveló algo que ninguna ejecución local había mostrado: **66 pruebas en verde en Windows y 23 en rojo en Linux**, sobre exactamente el mismo código.

Las 23 que caían eran precisamente las que sustituían el SDK de Firebase con dobles de módulo. El síntoma era siempre idéntico: la lista de llamadas registradas aparecía vacía y el valor preparado con `mockResolvedValue()` no surtía efecto, como si el doble no existiera.

5.1 La causa

El compilador de Angular empaqueta todos los archivos de prueba juntos y reparte los módulos compartidos en bloques según su propio criterio, que no es idéntico en las dos plataformas. En Linux, el bloque que recibió el doble no era el mismo que el servicio estaba usando: **la prueba vigilaba una copia del doble mientras el servicio llamaba a otra.**

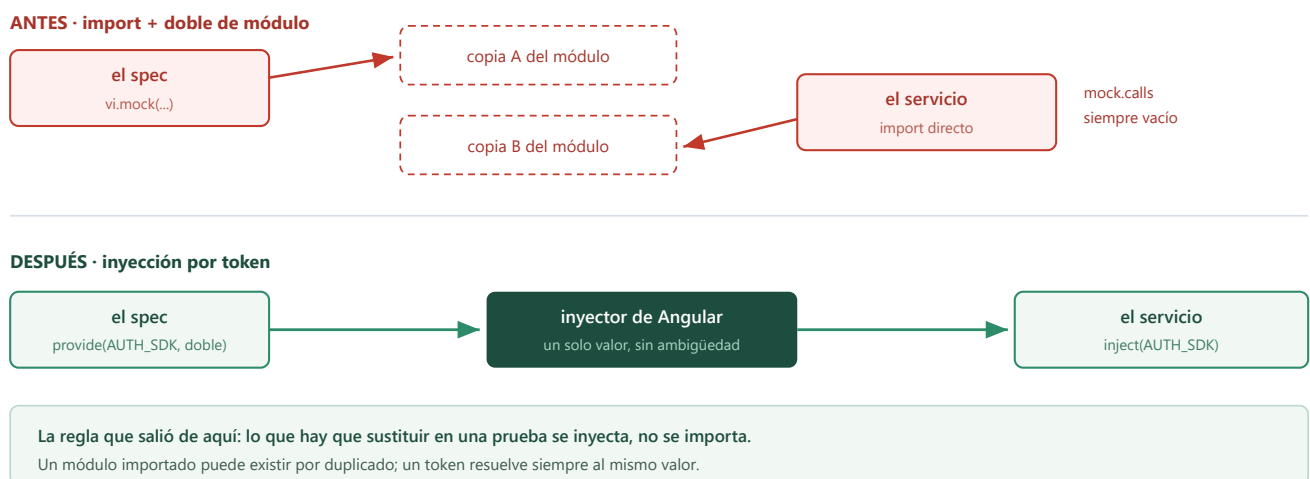


Figura 2 — El mismo código antes y después del cambio.

5.2 Lo grave no era el CI en rojo

Esas 23 pruebas tampoco verificaban nada en local. Pasaban por cómo cayó el reparto de bloques en Windows, no porque el código fuera correcto. Un fallo real en cualquiera de esos servicios habría pasado inadvertido con la suite entera en verde. El CI no introdujo el problema: lo hizo visible al ejecutar lo mismo en otra plataforma.

La solución fue dejar de importar lo que las pruebas necesitan sustituir. Las funciones del SDK pasan a viajar en dos tokens de inyección, y el tipo de cada campo se toma de la función real con `typeof`: si el SDK cambia una firma, el proyecto deja de compilar en lugar de mentir en silencio.

```
export interface FirestoreSdk {
  addDoc:      typeof addDoc;
  collection:  typeof collection;
  getDocs:     typeof getDocs;
  onSnapshot:  typeof onSnapshot;
  where:       typeof where;
  // ...solo los trozos que la aplicación llama de verdad
}

export const FIRESTORE_SDK = new InjectionToken<FirestoreSdk>('FIRESTORE_SDK');
```

Un único archivo, `app.config.ts`, importa los valores reales del SDK y los entrega. El resto de la aplicación solo conoce los tokens.

6. Pruebas automatizadas

Las 66 pruebas corren con **Vitest**, el runner que Angular 21 trae por defecto, y ninguna abre una conexión: los tokens reciben dobles, así que la suite termina en segundos y no necesita credenciales. Cinco archivos concentran el grueso de la verificación; el resto comprueba que cada componente se construye sin romperse.

Archivo	Qué valida	N.º
<code>auth.spec.ts</code>	Registro, entrada, salida y la traducción al español de cada error de Firebase	15
<code>expense-form.spec.ts</code>	Signal Forms: líneas dinámicas, validación cruzada contra el total y la tolerancia de 0.01	12
<code>budget-list.spec.ts</code>	Presupuesto duplicado (categoría + mes) y cruce de cada presupuesto con lo realmente gastado	11
<code>category.spec.ts</code>	<code>nameExists()</code> y el alta de categorías	8
<code>auth-guard · guest-guard</code>	Con sesión, sin sesión y con la sesión aún sin restaurar	8
Otros 12 archivos	Construcción de cada componente, resolver y guard de cambios sin guardar	12
Total — 18 archivos		66

```
$ npm test -- --watch=false

RUN v4.1.10 expense-tracker

Test Files  18 passed (18)
Tests       66 passed (66)
Duration   16.65s
```

Figura 3 — Salida real de la suite (Vitest 4.1.10).

La prueba más valiosa del conjunto es la del guard con la sesión todavía sin restaurar. Firebase resuelve el estado de la sesión de forma asíncrona, y un guard que decide antes de recibir la respuesta expulsa al login a usuarios válidos cada vez que recargan. En desarrollo ese parpadeo es casi invisible; la prueba lo fija.

7. Integración continua y despliegue

Un solo trabajo: formato, las 66 pruebas y un build de producción, en cada *push* y en cada *pull request*. Como nada toca la red, no hay credenciales que administrar ni riesgo de que dos ejecuciones simultáneas se estorben.

Este pipeline **no despliega**, y es una diferencia deliberada respecto de *reading-shelf*, donde el despliegue cuelga de las pruebas end-to-end. Allí existen pruebas contra el Firebase real que justifican montar esa puerta; aquí el coste de mantener un token de despliegue no se compensa. Dejarlo escrito en el propio workflow evita que la diferencia parezca un olvido.

El despliegue lo realiza la integración de Vercel. La configuración vive en `vercel.json` para que quede versionada y pueda explicarse en un commit, en lugar de residir en un formulario web. Al ser una SPA, incluye una reescritura que envía cualquier ruta no encontrada a `index.html`: sin ella, recargar en `/budgets` daría 404, porque ese archivo no existe en el disco — la ruta solo existe dentro del router de Angular.

8. Limitaciones conocidas

- `nameExists()` distingue mayúsculas: *comida* y *Comida* conviven como categorías distintas. Corregirlo exige un campo `nameLower`, que obliga a tocar el `hasOnly` de las reglas y volver a publicarlas.
- La unicidad de categoría + mes en los presupuestos se valida en el cliente y no puede trasladarse a las reglas: no existe forma de expresar «que no exista ningún otro igual» sobre una colección sin leerla entera.
- El detalle de cada línea de gasto se valida en forma superficial en las reglas; el contenido concreto se confía al cliente, que valida antes de escribir.
- El historial del repositorio agrupa demasiado trabajo por commit. No hay nada que corregir hacia atrás, pero es una práctica que conviene cambiar en adelante.

9. Conclusiones

Signal Forms resolvió con naturalidad lo que en formularios tradicionales resulta incómodo: un array de campos que cambia de tamaño y una regla que depende de dos partes distintas del modelo. Declarar las validaciones sobre la forma del modelo, en lugar de construirlas campo por campo, hace que el formulario se lea como el enunciado del problema.

El modelo de datos demostró que colocar la propiedad en un campo, y no en la ruta del documento, traslada trabajo a las reglas: hay que validar el dueño en las cuatro operaciones y, además, impedir que cambie. Es una decisión legítima, pero no gratuita.

La lección de mayor alcance vino del fallo de la sección 5. Un conjunto de pruebas puede estar entero en verde y no verificar nada, y la única razón por la que esto salió a la luz fue ejecutar la misma suite en otra plataforma. De ahí sale una regla que ya se aplica desde el primer día en los proyectos siguientes: **lo que hay que sustituir en una prueba se inyecta, no se importa.**

10. Método de trabajo

El proyecto se construyó con asistencia de herramientas de IA, y el repositorio lo declara: `CLAUDE.md`, versionado junto al código, recoge las convenciones e instrucciones con las que se trabajó.

El episodio de la sección 5 muestra dónde está el límite de esa asistencia. Las 66 pruebas estaban en verde y el código parecía sano; nada en él pedía revisión. Lo que destapó el problema fue ejecutar la misma suite en otra plataforma, leer con atención un fallo incómodo en lugar de reintentarlo, y llegar hasta la causa —el reparto de módulos en bloques— en vez de quedarse en el síntoma. La consecuencia fue rehacer cómo toda la aplicación recibe sus dependencias.

Una herramienta no decide que algo que funciona está mal. Esa es la parte que no se delega.